

Electricity Consumption Prediction using Regression & Machine Learning

Submitted To:

Dr. Kusum Bharti

Assistant Professor (Grade-I)

Department of Information Technology

Submitted By:

Gyanendra Mohan Patel

Roll No: 25201807

M.Tech Data Analytics

NIT Jalandhar

November 2025

Contents

1	Introduction	3
2	Dataset Description	3
3	Data Generation	3
4	Exploratory Data Analysis (EDA)	4
5	Data Preprocessing	4
6	Regression Models	5
7	Model Selection	6
8	Final Evaluation	6
9	Residual Analysis	6
10	Model Saving and Deployment	6
11	Custom Prediction Example	6
12	Conclusion	7
13	Future Work	7

1 Introduction

This project focuses on predicting electricity consumption (in kWh) using synthetic data generated from environmental and household usage parameters such as temperature, humidity, appliances in use, time of day, and day of the week. Two machine learning models—Linear Regression and Random Forest—were implemented and evaluated to compare their performance.

2 Dataset Description

- **Total Records:** 500 (synthetic, time-series-like)
- **Features:**
 - Temperature: 5–45°C
 - Humidity: 20–90%
 - TimeOfDay: 0–23
 - DayOfWeek: 1–7
 - ActiveAppliances: 1–20
 - EnergyUsage_kWh: Target variable
- No missing values were found.

3 Data Generation

```
1 import numpy as np
2 import pandas as pd
3
4 np.random.seed(42)
5 data_size = 500
6
7 temperature = np.random.uniform(5, 45, data_size)
8 time_of_day = np.random.randint(0, 24, data_size)
9 active_appliances = np.random.randint(1, 20, data_size)
10 humidity = np.random.uniform(20, 90, data_size)
11 day_of_week = np.random.randint(1, 8, data_size)
12
13 energy_usage = (
14     0.8*temperature +
15     1.4*time_of_day +
16     2.2*active_appliances -
17     0.3*humidity +
18     3*np.sin(time_of_day/24 * 2 * np.pi) +
19     np.random.normal(0, 6, data_size)
20 )
21
```

```

22 df = pd.DataFrame({
23     "Temperature": temperature,
24     "Humidity": humidity,
25     "TimeOfDay": time_of_day,
26     "DayOfWeek": day_of_week,
27     "ActiveAppliances": active_appliances,
28     "EnergyUsage_kWh": energy_usage
29 })

```

4 Exploratory Data Analysis (EDA)

The EDA reveals:

- Strong positive correlation with:
 - TimeOfDay
 - ActiveAppliances
 - Temperature
- Mild negative correlation with Humidity.
- No missing values or anomalies.
- Energy usage distribution appears near-normal.

5 Data Preprocessing

- 70% training, 15% validation, 15% testing.
- Standardization using StandardScaler.

```

1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3
4 X = df.drop("EnergyUsage_kWh", axis=1)
5 y = df["EnergyUsage_kWh"]
6
7 X_train, X_temp, y_train, y_temp = train_test_split(
8     X, y, test_size=0.3, random_state=42)
9
10 X_val, X_test, y_val, y_test = train_test_split(
11     X_temp, y_temp, test_size=0.5, random_state=42)
12
13 scaler = StandardScaler()
14 X_train_scaled = scaler.fit_transform(X_train)
15 X_val_scaled = scaler.transform(X_val)
16 X_test_scaled = scaler.transform(X_test)

```

6 Regression Models

1. Linear Regression

```
1 from sklearn.linear_model import LinearRegression
2 from sklearn.metrics import mean_absolute_error, mean_squared_error
3 import numpy as np
4
5 lr = LinearRegression()
6 lr.fit(X_train_scaled, y_train)
7 lr_pred = lr.predict(X_val_scaled)
8
9 print("Linear Regression MAE:", mean_absolute_error(y_val, lr_pred))
10 print("Linear Regression RMSE:", np.sqrt(mean_squared_error(y_val, lr_pred)))
```

Results:

- MAE: 4.85
- RMSE: 6.13

2. Random Forest

```
1 from sklearn.ensemble import RandomForestRegressor
2 from sklearn.model_selection import GridSearchCV
3
4 rf = RandomForestRegressor()
5 params = {
6     "n_estimators": [100, 200],
7     "max_depth": [5, 10, None],
8     "min_samples_split": [2, 5]
9 }
10
11 grid = GridSearchCV(rf, params, cv=3,
12                     scoring="neg_mean_squared_error",
13                     n_jobs=-1)
14 grid.fit(X_train, y_train)
15
16 best_rf = grid.best_estimator_
17 rf_pred = best_rf.predict(X_val)
18
19 print("Random Forest MAE:", mean_absolute_error(y_val, rf_pred))
20 print("Random Forest RMSE:", np.sqrt(mean_squared_error(y_val, rf_pred)))
21 print("Best Hyperparameters:", grid.best_params_)
```

Results:

- MAE: 6.53
- RMSE: 8.56

7 Model Selection

Linear Regression outperformed Random Forest, so it was selected as the final model.

8 Final Evaluation

```
1 if best_model == lr:
2     final_pred = best_model.predict(X_test_scaled)
3 else:
4     final_pred = best_model.predict(X_test)
5
6 print("Final MAE:", mean_absolute_error(y_test, final_pred))
7 print("Final RMSE:", np.sqrt(mean_squared_error(y_test, final_pred)))
```

Model	MAE	RMSE
Linear Regression	4.66	5.78
Random Forest	6.53	8.56

9 Residual Analysis

Residuals (Actual - Predicted) are centered around zero, indicating good model performance.

10 Model Saving and Deployment

```
1 import joblib
2
3 joblib.dump(best_model, "energy_model.pkl")
4 joblib.dump(scaler, "feature_scaler.pkl")
5 print("Model and scaler saved successfully!")
```

11 Custom Prediction Example

```
1 import pandas as pd
2 from sklearn.linear_model import LinearRegression
3 import joblib
4
5 model = joblib.load("energy_model.pkl")
6 scaler = joblib.load("feature_scaler.pkl")
7
8 example = pd.DataFrame({
9     "Temperature": [32],
10    "Humidity": [55],
11    "TimeOfDay": [18],
12    "DayOfWeek": [4],
13    "ActiveAppliances": [8]
```

```
14 })
15
16 if isinstance(model, LinearRegression):
17     example_scaled = scaler.transform(example)
18     result = model.predict(example_scaled)
19 else:
20     result = model.predict(example)
21
22 print("Predicted Energy Usage (kWh):", result[0])
```

Predicted energy usage for this custom input: **50.22** kWh

12 Conclusion

- Linear Regression performed best.
- Key features: Temperature, TimeOfDay, ActiveAppliances.
- Model ready for scaling to real-world datasets.

13 Future Work

- Use real household electricity data.
- Explore deep learning approaches.
- Create interactive dashboard or API.